

Application Platform Delivery Foundations

Version 1, 2026-07-10

Introduction

Welcome to this quick course on the ***Application Platform Delivery Foundations***. This course is the **first** in a series of **six** courses about **Application Platform Delivery**:

- Foundations: Developer Experience and GitOps Workflows (*This course*)
- *Cloud-Native Development with Dev Spaces*
- *Software Factory and CI/CD*
- *Application Modernization*
- *Developer Hub and Platform Engineering*
- *Trusted Software Supply Chain (TSSC)*

What's in this course?

Enable learners to understand the principles of Developer Experience, Platform Engineering, and GitOps-based software delivery while preparing the environment and skills required for subsequent Software Factory implementation labs.

Duration: 2 hours

Contributors

The PTL team acknowledges the valuable contributions of the following Red Hat associates:

- .. (SME)
- .. (SME)
- Yogita Soni (Content Architect)
- Anna Swicegood (Learning Product Manager)

Objectives

On completing this course, you should be able to:

Based on the section index, here are concise learning objectives:

Learning Objectives

By the end of this module, you will be able to:

- Explain the challenges that Platform Engineering addresses and its role in improving developer experience.
- Describe the core principles of GitOps and how it enables consistent, automated application delivery.

- Identify the components of the lab environment and verify that it is ready for the exercises.
- Understand common Git branching strategies and determine when to use each approach.
- Validate essential Git operations using fundamental Git commands.
- Explain the GitHub Flow workflow for collaborative software development.
- Apply Git and GitHub best practices through hands-on lab exercises involving branching, committing, merging, and pull requests.
- Summarize the key concepts of Developer Experience, Platform Engineering, Git, and GitOps workflows.

Prerequisites

This course assumes that you have the following prior experience:

1. [Red Hat OpenShift Container Platform \(OCP\) Technical Overview](#) or Kubernetes Basics

Lab Environment

Click the [RHDP](#) link above, log in to RHDP, and click **Order** to request a new [Service Enablement: App Platform Software Factory](#). On the order page for the catalog item, do the following:

- **Activity:** Select **Practice/Enablement**
- **Purpose:** Select **Learning about the Product**
- **Salesforce ID:** If you have an opportunity ID, enter it into this field, otherwise select **Project**, and then enter "Learning about RHCL" in the text field. You will see a warning message that a Salesforce ID is required, but you can safely ignore it.
- **Features:** Select **Enable Let's Encrypt**
- **Region:** Select the AWS region closest to your location
- **User count:** **1**
- **Control Plane Count:** **1**
- **OpenShift Version:** **4.16**
- **Control Plane Instance Type:** **m6a.4xlarge**
- Select the checkbox at the bottom of the page to confirm that lab costs will be charged to your cost center, and click **Order**.



The lab environment will take an hour to provision. You can check the status and details of the OpenShift cluster from the **Services** page of RHDP. The classroom is available for 48 hours initially. You can extend the duration of classroom availability by clicking **Help > Get Technical Help** in RHDP, and then requesting an extension by opening a ServiceNow ticket with the RHDP team.

RHDP Portal Links

- [RedHat Associates](#)
- [RedHat Partners](#)

Understanding the Need for Platform Engineering

The Developer's Struggle

- Imagine you are a developer who has just finished building a *new* feature that customers have been eagerly waiting for. The code works perfectly on your machine, but now comes the difficult part—getting it into production.

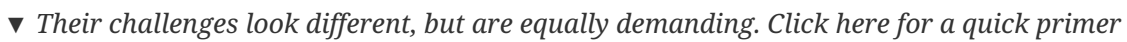


▼ *Questions begin to pile up: Click here for a quick primer*

- How do I onboard my application onto the platform?
 - Which tools, frameworks, and libraries should I use, and are they approved by the organization?
 - How do I deploy my code quickly and receive feedback faster?
 - What changes are required to modernize my application while continuing to deliver business value?
 - How can I ensure my application meets security and compliance requirements?
 - If I need new capabilities from the platform, how do I request them?
- Instead of focusing on delivering business value, developers often spend significant time navigating tooling decisions, understanding platform processes, modernizing applications, and coordinating with multiple teams. As complexity grows, innovation slows and delivery becomes more difficult.

The Platform Team's Struggle

- On the other side of the organization sits the platform team, responsible for building and operating the platforms that developers rely on every day.



- # DevOps Apocalypse

- ▼ *Click here for a quick overview of the challenges:*

- 5

- The result was slower delivery, increased frustration, and reduced innovation across the organization.



- Out of these challenges emerged Platform Engineering.
- Platform Engineering applies a product mindset to internal platforms, treating developers as customers and the platform itself as a product designed to solve their problems.
- By providing self-service capabilities, golden paths, and built-in security and governance, platform engineering enables developers to focus on what they do best—building applications that deliver business value.
- Instead of choosing between speed and control, organizations can achieve both.
- Platform Engineering accelerates software delivery through standardization, automation, and a relentless focus on developer experience.

What is DevEx and why is it a solution?

Developer Experience (DevEx) is the practice that makes Platform Engineering successful.

▼ *Click here to know: What is it?*

- Developer Experience (DevEx) is the sum of all factors that affect a developer's work throughout the Software Development Lifecycle (SDLC).
- It encompasses the tools, processes, platforms, environments, and organizational practices that influence how effectively developers can build, test, deploy, and operate software.
- The goal of DevEx is to reduce friction and enable developers to focus on delivering business value.

▼ *Click here to know: Why does it matter?*

- Platform Engineering succeeds when platforms are designed around the needs of developers.
- Developer feedback is critical and should flow continuously between developers and platform teams.
- This feedback helps identify friction points, prioritize improvements, and guide the evolution of the platform.
- By treating developers as customers, platform teams can create solutions that are easier to adopt and more effective to use.

▼ *Click here to know: Business Outcomes*

- Prioritizing Developer Experience makes it easier to develop, test, deploy, and operate software.
- Reduced friction leads to higher developer productivity and satisfaction.
- Faster feedback cycles enable teams to innovate more quickly and deliver value sooner.
- The result is improved software quality, faster innovation, and better business outcomes.

Benefits of a Developer-Centric Culture

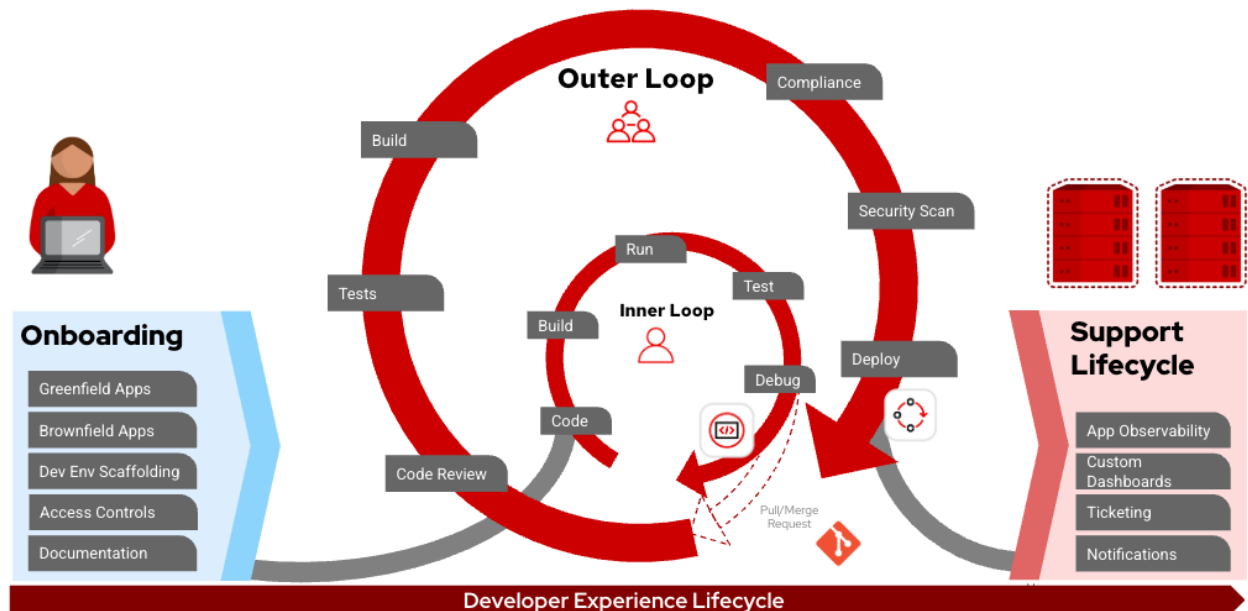
- But you might be wondering why should companies care about measuring and improving developer productivity? *Well, it is simple!*



- Companies that care and have a great developer centric culture report a massive boost in revenue - a 400% percent increase in revenue compared to companies that don't have developer centric cultures.
- A Developer Centric culture delivers happiness!
 - Happier employees, in turn make companies more productive and result in that 4x revenue increase.
- If you get a chance, definitely read the [McKinsey article](#). It is fascinating!
- Another big benefit of developer centric companies is employee retention. Happy, productive employees want to stay longer and that in turn drives more revenue.

Developer Experience Lifecycle

- Imagine a developer joining a platform for the first time.



- They begin with **onboarding**, gaining access to the tools, environments, and resources needed to get started.
- From there, they enter the **Inner Loop**—writing code, building, testing, and debugging applications.
- Once changes are ready, they move into the **Outer Loop**, where code is reviewed, validated, secured, and deployed through automated delivery pipelines.
- After deployment, applications enter the **Support Lifecycle**, where monitoring, observability, and operational processes help keep them running reliably.
- Together, these stages form the **Developer Experience Lifecycle**.
- Without a right integration between both loops (**Inner and Outer**), based on right processes supported by the Platform Teams, the Developer Experience will be impacted with low rates of productivity, poor performance, and low adoption rates of the platform created supposed to help or support them.
- This complexity must be addressed accordingly.

Accelerating Developer Productivity and Security

- *Imagine an organization that has invested in a modern application platform, yet developers still struggle with onboarding, deployment processes, tooling decisions, and slow feedback cycles. At the same time, platform teams are working hard to provide capabilities that developers may not be fully adopting or utilizing.*
- This is where a **Developer Experience Assessment** helps.
- Through a collaborative engagement, Red Hat Consulting works closely with platform teams and developers to understand how software is built, delivered, and operated across the organization.
- Together, they identify pain points, uncover opportunities for improvement, and align platform capabilities with developer needs.
- The outcome is a prioritized roadmap that helps improve:

- Developer productivity
- Strengthen security
- Accelerate platform adoption.
- The roadmap also provides guidance for building key capabilities such as:
 - **An Internal Developer Portal (IDP)**
 - **A Trusted Software Supply Chain (TSSC)**
 - **Self-service developer experiences**
- *Success requires more than technology alone.*
- It depends on active collaboration, hands-on workshops, and open discussions where platform teams and developers work together to shape the future developer experience.

Requirements & Environment Setup

Overview

This initial setup session establishes the foundational environment used throughout the remaining 5 modules in this series of course. Ensure all requirements are met before proceeding.

Requirements

OpenShift Cluster

You will need an OpenShift Container Platform cluster for the labs. Order a [Red Hat OpenShift Container Platform Cluster \(Multi-Cloud\)](#) from the Red Hat Demo Platform with the following options:

Option	Value
Activity	Practice / Enablement
Purpose	Conduct internal training/enablement
Cloud Provider	cnv
OpenShift Version	4.21
Cluster Size	multinode
OpenShift Worker Count	3
Worker CPU	16
Worker memory	32Gi
Create Users on Cluster	Unchecked
Enable OpenShift Lightspeed	Unchecked



Order your cluster **before** the lab begins — provisioning can take up to 60 minutes. Once provisioning is complete you will receive an email with your cluster

credentials and console URL.

Git Server Access

- Access to the provided Git server instance
- Repository permissions confirmed

Laptop Setup

- Git client installed and configured
- Terminal or IDE with Git integration
- SSH key or personal access token configured for the Git server

Environment Verification

Verify your OpenShift cluster is accessible:

```
oc login -u admin --server={openshift_api_url}
oc whoami
oc get nodes
```

Verify your Git client is installed:

```
git --version
```

Confirm you can authenticate to the Git server:

```
# Get GitLab URL dynamically
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"

# Clone the repository
git clone ${GITLAB_URL}/software-factory/etx_app_base_app.git
```

Next Steps

Once your environment is verified, proceed to [Branching Strategies](#).

Branching Strategies

Overview

In this session you will validate your Git setup and explore common Git branching strategies. Understanding branching models is essential for the CI/CD and GitOps workflows.

This session includes:

- Git basics validation
- Branching models presentation
- Hands-on branching exercise

Git Basics Validation

Before diving into branching strategies, let's verify your Git environment is working correctly.

Run the commands in the OpenShift Web Terminal provided by the workshop environment. The Web Terminal includes the required CLI tools and avoids local workstation prerequisites.

Step 1: Access GitLab

First, verify you can access GitLab through your browser.

Get the GitLab URL and open it in your browser:

```
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o  
jsonpath='{.items[0].spec.host}')"  
echo "Open this URL in your browser: $GITLAB_URL"
```

Login Process

1. Click the **"ETX Lab SSO"** button (SSO login)
2. You'll be redirected to Keycloak authentication
3. Login with workshop credentials:
 - Username: `{user}`
 - Password: `{password}`



Workshop Users Available:

- `platform-admin` - Administrative access
- `platform-user` - Standard user access
- `platform-developer` - Developer access

All users use password: **openshift**

After login, you should see the GitLab interface with the **software-factory** group.

Step 2: Clone the Workshop Repository

Now clone the ETX App Base application repository using Git CLI.

First, get the GitLab URL:

```
# Get GitLab URL dynamically
GITLAB_URL="https://$(oc get route -n gitlab -l app=webservice -o
jsonpath='{.items[0].spec.host}')"
echo "GitLab URL: $GITLAB_URL"
```

Then clone the repository:

```
mkdir -p ~/workshop
cd ~/workshop
git clone ${GITLAB_URL}/software-factory/etx_app_base_app.git
cd etx_app_base_app
```



Authentication for Git CLI:

When prompted for credentials during **git clone**, use your assigned workshop credentials or create a personal access token from GitLab: Settings → Access Tokens → Create token with **read_repository** scope

Expected Output

```
Cloning into 'etx_app_base_app'...
remote: Enumerating objects: 150, done.
remote: Counting objects: 100% (150/150), done.
Receiving objects: 100% (150/150), done.
```

Step 3: Verify Basic Git Commands

```
# Check repository status
git status

# View current branch
git branch

# View recent commits
git log --oneline -5
```

What You Should See

- Current branch: `main`
- Working directory: clean
- Recent commit history displayed



If any of these commands fail, verify:

- Git client is installed: `git --version`
- Git is configured in the repository: `git config --list --local`
- You have network access to GitLab

Step 4: Configure Git (If Needed)

If this is your first time using Git on this system:

```
git config user.name "Platform Developer"
git config user.email "platform-developer@example.com"
```



These commands configure Git only for the current repository. Use `git config --global` only on a workstation or bastion environment that you own and intend to reuse. Replace the name and email with your actual identity.

Validation Complete - You're ready to work with branches.

Branching Models

Different teams use different branching strategies based on their release cadence, team size, and deployment practices.

Trunk-Based Development

[Trunk-Based Development] | <https://trunkbaseddevelopment.com/trunk1b.png>

Trunk-Based Development is a lightweight strategy where developers integrate small, frequent changes into `main` to keep the codebase continuously releasable and deployable.

Characteristics:

- All developers commit to a single `main` branch
- Short-lived feature branches (hours to days)
- Frequent integration (multiple times per day)
- Continuous Integration runs on every commit
- Feature flags for incomplete features

Best suited for:

- Teams practicing Continuous Delivery
- Small to medium teams (2-10 developers)
- Projects with automated testing
- Cloud-native applications

Example workflow:

```
# Create short-lived feature branch
git checkout -b feature/add-validation

# Work, commit frequently
git commit -am "add email validation"

# Merge back quickly (same day)
git checkout main
git merge feature/add-validation
git push origin main
```

Understanding Git/GitHub Flow

GitFlow

[GitFlow] | <https://nvie.com/img/git-model@2x.png>

GitFlow is a structured branching model that separates development and release work through dedicated branch types, making it suitable for teams with planned release cycles.

Characteristics:

- Multiple long-lived branches: **main**, **develop**, **release/***, **hotfix/***
- Feature branches created from **develop**
- Release branches for production preparation
- Hotfix branches for urgent production fixes
- Well-defined promotion path

Best suited for:

- Teams with scheduled release cycles (monthly, quarterly)
- Larger teams with dedicated release managers
- Projects requiring multiple supported versions
- Enterprise applications with formal change control

Branch structure:

- **main** - Production-ready code

- **develop** - Integration branch for features
- **feature/*** - New features (from develop)
- **release/*** - Preparing next production release
- **hotfix/*** - Emergency production fixes (from main)

GitHub Flow

[GitHub Flow] |

https://miro.medium.com/v2/resize:fit:720/format:webp/1*S4VJPf8Mrg3Gn5cR7JK62w.png

GitHub Flow is a lightweight branch-and-pull-request workflow focused on short-lived branches and frequent merges into **main**.

Characteristics:

- Simplified model with **main** + feature branches
- Pull Requests for all changes
- CI/CD runs on every PR
- Deploy from **main** after merge
- No separate develop branch

Best suited for:

- Teams practicing Continuous Deployment
- Web applications deployed frequently
- Projects with strong automated testing
- Open source projects

Example workflow:

```
# Create feature branch
git checkout -b feature/user-profile

# Work and push frequently
git commit -am "add user profile page"
git push origin feature/user-profile

# Open Pull Request in GitLab
# CI runs automatically
# Deploy to staging for review
# Merge to main after approval
# Deploy to production automatically
```


Choosing the Right Model

Model	Release Frequency	Team Size	Complexity
Trunk-Based	Continuous	Small-Medium	Low
GitFlow	Scheduled	Medium-Large	High
GitHub Flow	Continuous	Any	Medium

For this workshop, we use **Trunk-Based Development** concepts:

- Short-lived feature branches
- Frequent integration
- CI/CD automation
- GitOps for deployment

Lab Exercise

Now let's practice the fundamental branching operations you'll use throughout Day 2 labs.

Exercise 1: Create a Feature Branch

Create a new branch for your changes:

```
# Ensure you're on main
git checkout main

# Create and switch to feature branch
git checkout -b feature/update-readme

# Verify you're on the new branch
git branch
```

Expected Output

```
* feature/update-readme
main
```

The asterisk (*) indicates your current branch.



Branch naming convention:

- **feature/** - New features
- **fix/** - Bug fixes
- **docs/** - Documentation changes
- **refactor/** - Code refactoring

Example: `feature/add-authentication`, `fix/login-bug`

Exercise 2: Make Changes and Commit

Make a simple change to the README:

```
# Add your name to the README
cat >> README.md << EOF

## Workshop Participant
- Name: PUT_YOUR_NAME_HERE
- Date: $(date +%Y-%m-%d)
EOF

# View what changed
git status
git diff README.md
```



Replace `PUT_YOUR_NAME_HERE` with your name.

Stage and commit your changes:

```
# Stage the file
git add README.md

# Commit with a descriptive message
git commit -m "docs: add workshop participant information"

# View your commit
git log --oneline -1
```



Commit Message Format

Using [conventional commit](#) format creates a commit message that is easy to understand and follow:

- **feat:** - New feature
- **fix:** - Bug fix
- **docs:** - Documentation
- **refactor:** - Code restructuring
- **test:** - Adding tests
- **chore:** - Miscellaneous changes

Example: `feat: add user authentication`

Exercise 3: Push to Remote

Push your branch to GitLab:

```
# Push to remote (first time for this branch)
git push -u origin feature/update-readme
```

Expected Output

```
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Writing objects: 100% (3/3), 312 bytes | 312.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0)
To {gitlab_url}/software-factory/etx_app_base_app
 * [new branch]      feature/update-readme -> feature/update-readme
```

Exercise 4: Verify in GitLab UI

1. Get the GitLab URL and open it in your browser:

```
echo "Open this URL in your browser: $GITLAB_URL"
```

2. Login with Keycloak (if not already logged in)
3. Navigate to: **Projects** → **software-factory** → **etx_app_base_app**
4. Click on **Code** → **Branches**
5. You should see your **feature/update-readme** branch

[GitLab Branches] | *gitlab-branches-view.png*

Exercise 5: Merge Your Changes

In a real workflow, you'd create a Merge Request for review. For this exercise, merge directly:

```
# Switch to main
git checkout main

# Merge your feature branch
git merge feature/update-readme
```

Expected Output (Fast-forward merge)

```
Updating a1b2c3d..e4f5g6h
Fast-forward
 README.md | 4 ++++
```

1 file changed, 4 insertions(+)



Fast-forward merge: Git simply moves the `main` pointer forward because no other changes were made to `main`. This is the simplest type of merge.

Push the merged changes:

```
git push origin main
```

Clean up the feature branch:

```
# Delete local branch
git branch -d feature/update-readme

# Delete remote branch
git push origin --delete feature/update-readme
```

Summary

In this session you:

- Validated your Git environment
- Learned three branching models:
 - Trunk-Based Development (continuous delivery)
 - GitFlow (scheduled releases)
 - GitHub Flow (continuous deployment)
- Practiced core Git operations:
 - Creating branches
 - Making commits
 - Pushing to remote
 - Merging changes

Key Takeaways

Branching enables parallel development:

- Multiple developers work on different features simultaneously
- Isolated changes until ready for integration
- Code review through merge requests
- CI/CD automation on every push

Different models suit different needs:

- Choose based on release cadence, team size, and deployment frequency
- This workshop uses Trunk-Based Development principles
- Day 2 labs will integrate branches with CI/CD pipelines

Best practices:

- Keep branches short-lived (hours to days)
 - Use descriptive branch names and commit messages
 - Push frequently to back up your work
 - Delete branches after merging
-